

Project Title: GitHub RAG Chatbot

<https://github.com/A-Narain/GitHub-RAG-Chatbot>

A RAG-based AI chatbot that lets you chat with any public GitHub repository using natural language.

Description:

Developers often waste hours reading code, navigating repositories, and exploring files; this AI-powered chatbot allows users to load any public GitHub repository and ask questions about it in natural language, receiving source-cited answers hence making codebase understanding easier and faster.

Tech Stack

- LangChain — RAG pipeline orchestration
- ChromaDB — Vector database for embeddings
- Gemini API — LLM + embeddings (gemini-2.5-flash)
- FastAPI — REST API backend
- React — Chatbot frontend UI
- PyGithub — GitHub API integration
- Python 3.9+ — Backend language

How it Works

1. Paste a GitHub repo URL
2. App fetches all code & docs via GitHub API
3. Files are split into chunks & embedded
4. You ask a question in natural language
5. Most relevant chunks are retrieved
6. Gemini generates a cited answer

Setup and Installation

Prerequisites

- Python 3.9+
- Node.js 16+
- Gemini API key (aistudio.google.com)
- GitHub Token (github.com/settings/tokens)

Backend Setup

```
git clone https://github.com/yourname/github-rag
cd github-rag
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

Configure .env

```
GOOGLE_API_KEY=your_gemini_key
GITHUB_TOKEN=your_github_token
CHROMA_PERSIST_DIR=C:\path\to\github_db
```

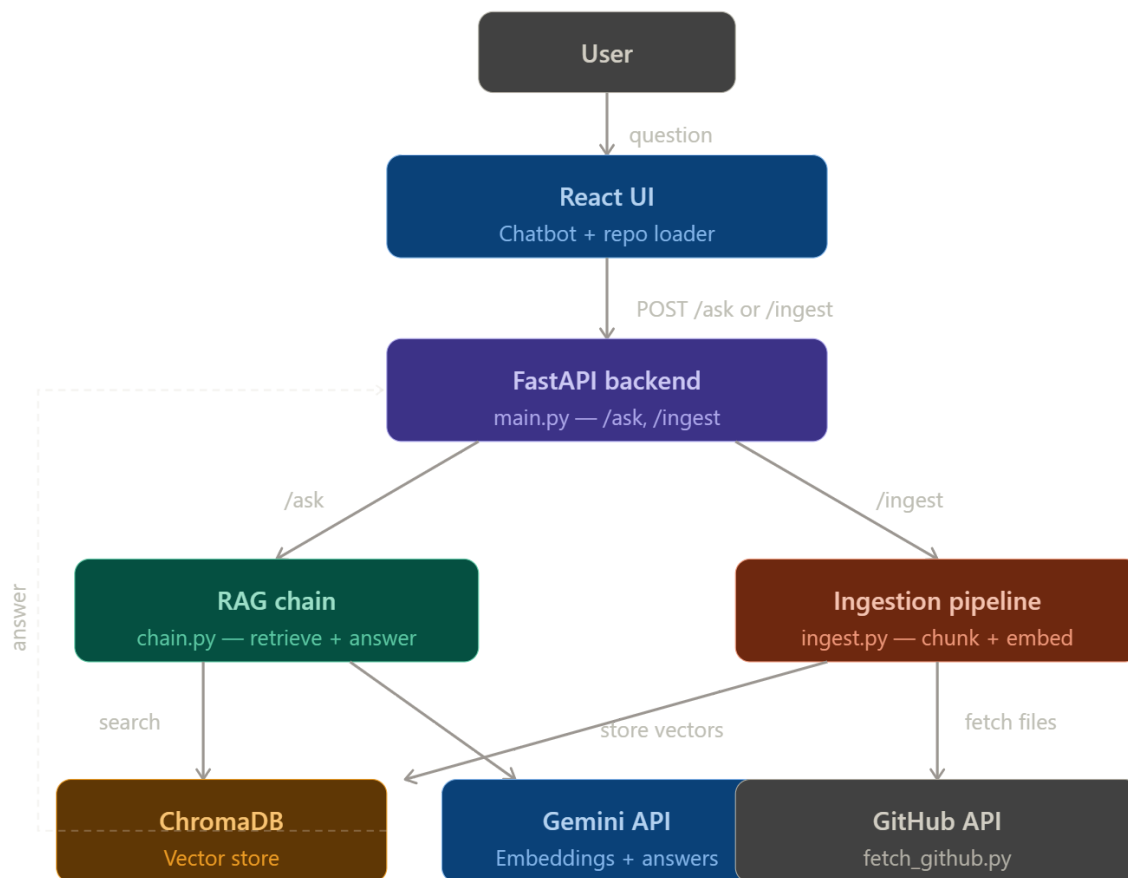
Run Backend

```
cd backend
uvicorn main:app --reload --port 8000
```

Frontend Setup

```
cd frontend
npm install
npm start
```

Project architecture



[Click any component to learn more](#)

API Endpoints

GET / → API status

GET /health → Health check

POST /ask Request: { "question": "What does this project do?" } Response: { "answer": "...", "sources": [...] }

POST /ingest Request: { "repo_url": "https://github.com/user/repo" } Response: { "success": true, "message": "..." }

Features

- Chat with any public GitHub repository
- Source citations with direct GitHub links
- Supports .py .md .js .ts .yaml and more
- MMR retrieval (no redundant answers)
- Auto-skips large files and irrelevant folders
- Swap repos dynamically from the UI
- REST API — can be integrated anywhere

Limitations

- Free tier: 1000 embedding requests/day
- Large repos take longer to ingest
- ChromaDB is local (not persistent across servers)
- Only supports public GitHub repositories

What I learned

- How RAG pipelines work end to end
- Vector embeddings and similarity search
- LangChain LCEL chain syntax
- ChromaDB vector store management
- FastAPI + React full stack integration
- GitHub API with PyGithub
- Rate limit handling strategies
- Prompt engineering for factual answers

Screenshots

The screenshot displays the GitHub RAG Chatbot interface. At the top, a blue header bar contains the chatbot's name, "GitHub RAG Chatbot", and the repository name, "mjhea0/flaskr-tdd". Below the header, there is a text input field for pasting a GitHub repository URL, with the example "https://github.com/pallets/flask" provided. A "Load Repo" button is located to the right of the input field.

The chat history shows the following interaction:

- Bot:** 🙋 Hi! I'm loaded with the mjhea0/flaskr-tdd repo. Ask me anything about it!
- User:** what does this project do ?
- Bot:** You'll be building a simple blogging app in this tutorial (File: README.md).
Sources: [README.md](#) [.github/FUNDING.yml](#) [project/app.py](#)
- User:** What routes does the Flask app have?
- Bot:** The Flask app has the following routes: `*`/`` (File: README.md) `*`/add`` (accepts POST requests) (File: README.md) `*`/logout`` (File: README.md)
Sources: [README.md](#) [project/app.py](#)
- User:** How do I run the tests?

At the bottom, there is a text input field for asking questions about the repository, with the placeholder text "Ask anything about the repo...", and a "Send" button to the right.



GitHub RAG Chatbot

mjhea0/flaskr-tdd

Paste GitHub repo URL e.g. <https://github.com/pallets/flask>

Load Repo

What is flaskr-tdd?

You'll be building a simple blogging app in this tutorial, which is referred to as "Flaskr-TDD" in the `README.md` file.

Sources: [README.md](#)

What database does this project use?

The project uses a Postgres database, as indicated by the provisioning instructions in `README.md` (`heroku addons:create heroku-postgresql:mini`). The `DATABASE_URL` environment variable, which is used to configure `SQLALCHEMY_DATABASE_URI` in `project/app.py`, points to a Postgres database. Additionally, the `requirements.txt` file lists `psycopg2-binary`, which is a PostgreSQL adapter. For testing purposes, the project uses SQLite, as configured in `tests/app_test.py` where `app.config["SQLALCHEMY_DATABASE_URI"]` is set to `sqlite:///BASE_DIR.joinpath(TEST_DB)`. SQLite is also used as a fallback for `SQLALCHEMY_DATABASE_URI` in `project/app.py` if the `DATABASE_URL` environment variable is not set.

Ask anything about the repo...

Send



GitHub RAG Chatbot

mjhea0/flaskr-tdd

Paste GitHub repo URL e.g. <https://github.com/pallets/flask>

Load Repo

How does user authentication work?

User authentication works as follows: 1. **Login**: * The `/login` route (defined in `README.md`) handles user login via GET and POST requests. * When a POST request is made, it checks the submitted `username` and `password` from the form against `app.config['USERNAME']` and `app.config['PASSWORD']`, respectively. * If the credentials match, `session['logged_in']` is set to `True`, a flash message "You were logged in" is displayed, and the user is redirected to the `index` page. * If the credentials do not match, an "Invalid username" or "Invalid password" error is displayed on the login page. * A helper function `login` in `tests/app_test.py` is used to simulate a login by posting data to the `/login` endpoint. 2. **Logout**: * The `/logout` route (defined in `README.md`) handles user logout. * It removes the `logged_in` key from the session (`session.pop('logged_in', None)`), displays a flash message "You were logged out", and redirects the user to the `index` page. 3. **Login Required Decorator**: * A `login_required` decorator (defined in `README.md`) can be used to protect routes. * This decorator checks if `session.get('logged_in')` is `True`. * If the user is not logged in, it flashes the message "Please log in." and returns a JSON response with `{'status': 0, 'message': 'Please log in.'}` and a 401 Unauthorized status code.

Ask anything about the repo...

Send

How are blog entries stored?

Blog entries are stored in a database using SQLAlchemy, as indicated by the `db.session.add(new_entry)` and `db.session.commit()` calls in the `add_entry` function (File: `README.md`). The `add_entry` function creates a `models.Post` object with the title and text from the request form and then adds it to the database session.

Sources: [project/templates/index.html](#) [README.md](#)

How is the database initialized?

The database is initialized by running the `create_db.py` script. In `create_db.py`:

```
python from project.app import app, db with app.app_context(): # create the database and the db table db.create_all() # commit the changes db.session.commit()
```

This script uses `db.create_all()` to create the database and its tables, and then `db.session.commit()` to commit these changes. The `README.md` also mentions that initially, SQLite will be used and will set up a single table with three fields: "id", "title", and "text". Later, it describes how to configure `SQLALCHEMY_DATABASE_URI` to use a Postgres database.

Sources: [create_db.py](#) [README.md](#) [project/models.py](#) [tests/app_test.py](#)

Ask anything about the repo...

Send

which python version is used ??

Python v3.12.0 is used (runtime.txt, README.md).

Sources: [runtime.txt](#) [README.md](#) [.github/workflows/main.yml](#) [project/templates/login.html](#)

What is the app.py file responsible for?

The `project/app.py` file is responsible for:

- * Initializing and configuring the Flask application, including loading configuration variables like `DATABASE` (from `README.md`).
- * Defining routes and handling HTTP requests, such as the root URL (`/`) which can return "Hello, World!" (from `README.md`) or search the database for entries and display them by rendering `index.html` (from `project/app.py`).
- * Running the Flask application when executed directly (from `project/app.py` and `README.md`).
- * Interacting with the database, specifically querying `models.Post` (from `project/app.py`).
- * Rendering HTML templates (e.g., `index.html`) (from `project/app.py`).
- * Implementing authentication logic, such as the `login_required` decorator for session management (from `project/app.py`).

Sources: [project/app.py](#) [README.md](#)

Ask anything about the repo...

Send



GitHub RAG Chatbot

 mjhea0/flaskr-tdd

Paste GitHub repo URL e.g. <https://github.com/pallets/flask>

Load Repo

 Sources: [README.md](#) [tests/app_test.py](#)

which python version is used ??

Python v3.12.0 is used (runtime.txt, README.md).

 Sources: [runtime.txt](#) [README.md](#) [.github/workflows/main.yml](#) [project/templates/login.html](#)

What is the app.py file responsible for?

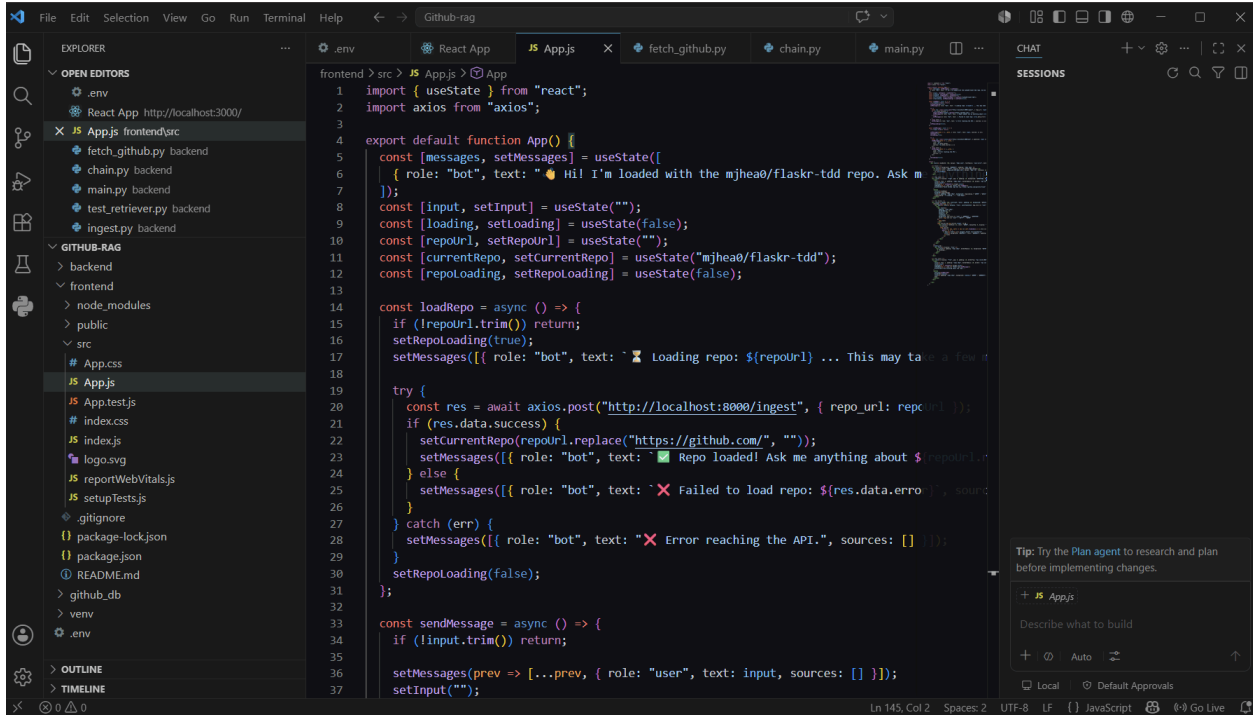
The `project/app.py` file is responsible for:

- * Initializing and configuring the Flask application, including loading configuration variables like `DATABASE` (from `README.md`).
- * Defining routes and handling HTTP requests, such as the root URL (`/`) which can return "Hello, World!" (from `README.md`) or search the database for entries and display them by rendering `index.html` (from `project/app.py`).
- * Running the Flask application when executed directly (from `project/app.py` and `README.md`).
- * Interacting with the database, specifically querying `models.Post` (from `project/app.py`).
- * Rendering HTML templates (e.g., `index.html`) (from `project/app.py`).
- * Implementing authentication logic, such as

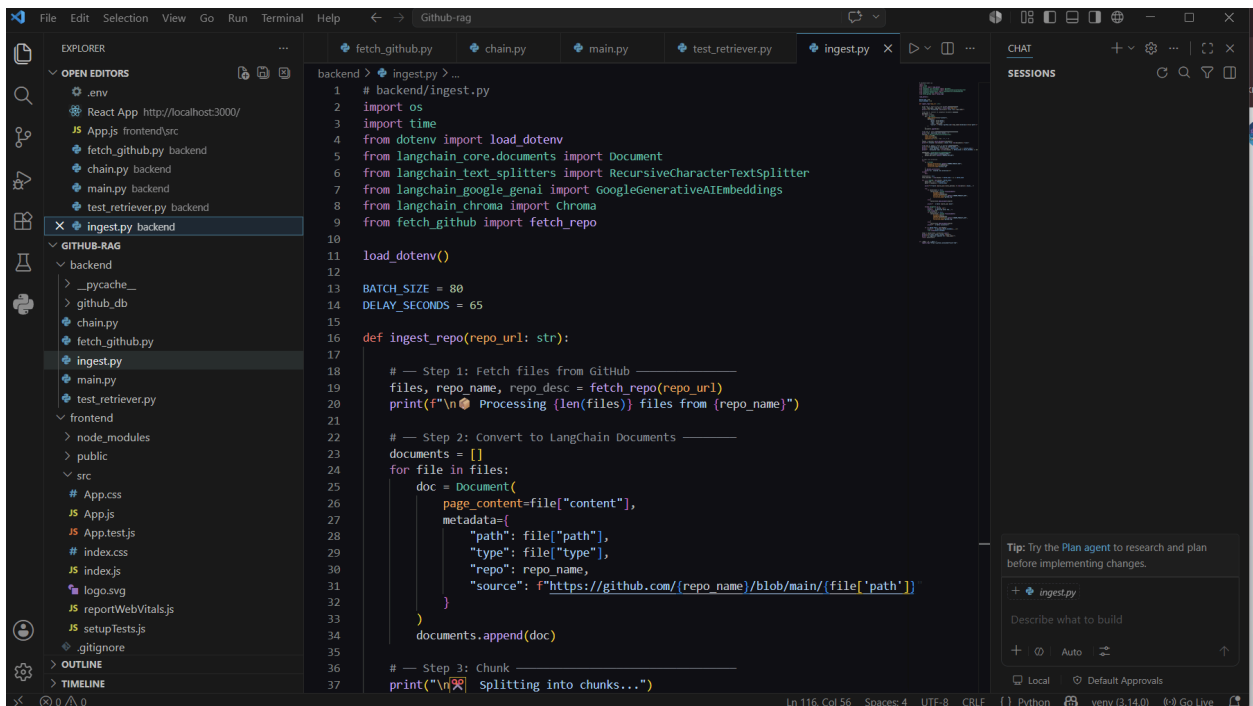
Ask anything about the repo...

Send

CODE SS



```
1 import { useState } from "react";
2 import axios from "axios";
3
4 export default function App() {
5   const [messages, setMessages] = useState([
6     { role: "bot", text: "👋 Hi! I'm loaded with the mjhea0/flaskr-tdd repo. Ask me anything." }
7   ]);
8   const [input, setInput] = useState("");
9   const [loading, setLoading] = useState(false);
10  const [repoUrl, setRepoUrl] = useState("");
11  const [currentRepo, setCurrentRepo] = useState("mjhea0/flaskr-tdd");
12  const [repoLoading, setRepoLoading] = useState(false);
13
14  const loadRepo = async () => {
15    if (!repoUrl.trim()) return;
16    setRepoLoading(true);
17    setMessages([
18      { role: "bot", text: "🔄 Loading repo: ${repoUrl} ... This may take a few seconds." }
19    ]);
20    try {
21      const res = await axios.post("http://localhost:8000/ingest", { repo_url: repoUrl });
22      if (res.data.success) {
23        setCurrentRepo(repoUrl.replace("https://github.com/", ""));
24        setMessages([
25          { role: "bot", text: "✅ Repo loaded! Ask me anything about ${currentRepo}." }
26        ]);
27      } else {
28        setMessages([
29          { role: "bot", text: "❌ Failed to load repo: ${res.data.error}. Sources: ${res.data.sources}." }
30        ]);
31      }
32    } catch (err) {
33      setMessages([
34        { role: "bot", text: "❌ Error reaching the API." }
35      ]);
36    }
37    setRepoLoading(false);
38  };
39
40  const sendMessage = async () => {
41    if (!input.trim()) return;
42    setMessages([
43      { role: "user", text: input, sources: [] }
44    ]);
45    setInput("");
46  };
47}
```



```
1 # backend/ingest.py
2 import os
3 import time
4 from dotenv import load_dotenv
5 from langchain_core.documents import Document
6 from langchain_text_splitters import RecursiveCharacterTextSplitter
7 from langchain_google_genai import GoogleGenerativeAIEmbeddings
8 from langchain_chroma import Chroma
9 from fetch_github import fetch_repo
10
11 load_dotenv()
12
13 BATCH_SIZE = 80
14 DELAY_SECONDS = 65
15
16 def ingest_repo(repo_url: str):
17   # Step 1: Fetch files from GitHub
18   files, repo_name, repo_desc = fetch_repo(repo_url)
19   print(f"📁 Processing {len(files)} files from {repo_name}")
20
21   # Step 2: Convert to LangChain Documents
22   documents = []
23   for file in files:
24     doc = Document(
25       page_content=file["content"],
26       metadata={
27         "path": file["path"],
28         "type": file["type"],
29         "repo": repo_name,
30         "source": f"https://github.com/{repo_name}/blob/main/{file['path']}"
31       }
32     )
33     documents.append(doc)
34
35   # Step 3: Chunk
36   print(f"🔪 Splitting into chunks...")
```

